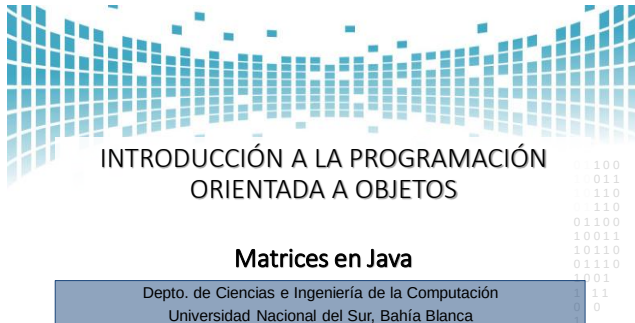




INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

Matrices en Java

Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

Videojuego Androides

En un Videojuego algunos de los personajes son **Androides**.
Los androides pueden aprender a hablar y a medida que el juego avanza pueden adquirir nuevos vocablos.

Cada android tiene una nave, cada nave tiene un **color** y una **cantidad de combustible** en el tanque.

Ambos atributos pueden aumentar o disminuir de acuerdo a un parámetro que puede ser positivo o negativo.

Los androides se organizan en una GRILLA de ANDROIDES

Videojuego Androide

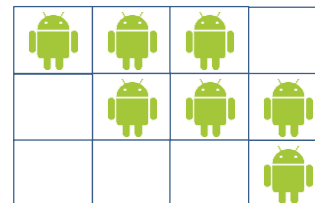
```

NaveEspacial
<<Atributos de clase>>
maxDeposito=1000 parsec=100
<<Atributos de instancia>>
color: char
combustible: entero
<<Constructores>>
NaveEspacial(o: char, c: entero)
NaveEspacial(o: char)
<<Comandos>>
establecerColor(o: char)
establecerCombustible(c: entero)
llenarDeposito()
copy(n:NaveEspacial)
<<Consultas>>
obtenerColor(): char
obtenerCombustible(): entero
obtenerAutonomia(): real
equals(n:NaveEspacial):boolean
clone():NaveEspacial
    
```

```

Androide
<<Atributos de instancia>>
vocablos: entero
nave: NaveEspacial
<<Constructor>>
Androide (n: NaveEspacial)
<<Comandos>>
establecerNave(n: NaveEspacial)
aprenderVocablos(v: entero)
copy(a: Androide)
<<Consultas>>
obtenerNave(): NaveEspacial
obtenerVocablos(): entero
clone(): Androide
equals(a: Androide): boolean
    
```

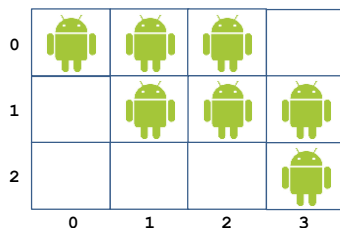
Grilla



```

Androide
<<Atributos de instancia>>
vocablos: entero
nave: NaveEspacial
NaveEspacial
<<Atributos de clase>>
maxDeposito, parsec: entero
<<Atributos de instancia>>
color: char
combustible: entero
    
```

Indices



```

01100
10011
10110
01110
01100
10011
10110
01110
1001
111
00
1
    
```

Declaración de la variable

En el momento que se declara la variable se establece el nombre y el tipo de los elementos.

```
Androide[] [] grilla;
```

```

01100
10011
10110
01110
01100
10011
10110
01110
1001
111
111
    
```

En Java los corchetes se utilizan para declarar un arreglo

Creación del arreglo de dos dimensiones

En el momento que se crea el arreglo se establece la cantidad de elementos.

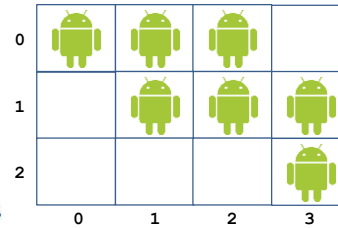
```
grilla = new Androide [nFil][nCol];
```

```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
```

Crearemos matrices con el mismo número de elementos en cada fila.

Subindicación

```
grilla [0]
```

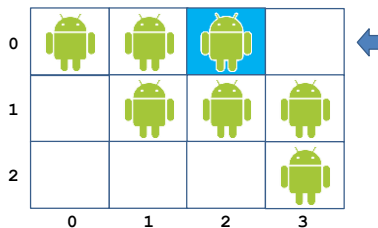


```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

En Java la primera fila se accede con el índice 0

Subindicación

```
grilla [0][2]
```

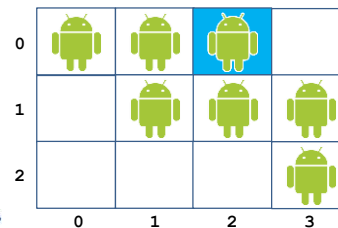


```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

`grilla [0][2]` es un objeto de clase `Androide`

Mensajes

```
grilla [0][2].obtenerVocablos()
```

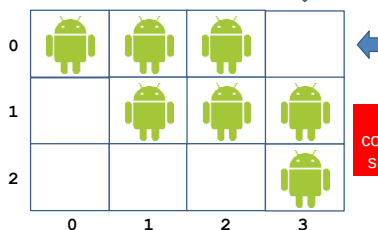


```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

`grilla [0][2]` puede recibir mensajes de acuerdo a su clase

Errores de ejecución

```
grilla [0][3].obtenerVocablos()
```

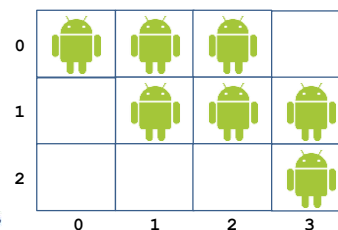


```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

Si la estructura no está completa debemos controlar si la referencia está ligada.

Errores de ejecución

```
grilla [0][4]
```

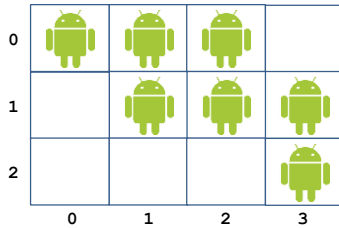


```
01100
10011
10110
01110
01100
10011
10110
10110
1001
1 11
0 0
1
```

Alguna de las clases debe asumir la responsabilidad de validar los índices.

El atributo length

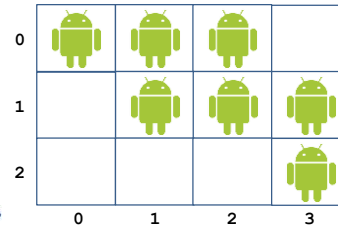
grilla.length



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

El atributo length

grilla [0].length



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

El número de componentes de la fila con índice 0

Patrones de algoritmos - Contar

```
Algoritmo contar
contador ← 0
para cada fila f
  para cada columna c
    si grilla[f,c] verifica la propiedad
      contador ← contador + 1
```

```
100
011
10110
01110
01100
10011
10110
01110
1001
1 11
```

Contar la cantidad de elementos que verifican una propiedad es un recorrido exhaustivo

Patrones de algoritmos - Búsqueda

```
Algoritmo buscar
Dato de entrada elemento
para cada fila f y mientras no encontremos el elemento
  para cada columna c y mientras no encontremos el elemento
    si grilla[f,c] es el elemento buscado
      encontramos elemento
```

```
100
011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
```

Diseñar un algoritmo nos permite identificar las estructuras de control sin detenernos en detalles sintácticos

Diseño

```
GrillaAndroides
<<atributos de instancia>>
grilla [][] Androide

<<Constructores>>
GrillaAndroides(nFil,nCol: entero)
<<Comandos>>
establecerAndroide(a:Androide,
f,c:entero)
<<Consultas>>
obtenerAndroide(f,c:entero): Androide
cantFilas(): entero
cantColumnas():entero
cantAndroides():entero
cantAndroidesFila(f:entero):entero
estaAndroide(a:Androide):boolean
alMenosN(n:entero, unaNave:
NaveEspacial):boolean
alMenosNCon(n:entero,
unaNave:NaveEspacial):boolean
```

- **GrillaAndroide(nFil,nCol: entero)**
Requiere nFil y nCol mayor a 0
- **establecerAndroide(a:Androide, f,c:entero)**
Requiere f,c válidos.
- **obtenerAndroide(f,c:entero): Androide**
Requiere f,c válidos.
- **cantAndroidesFila(f:entero):entero**
Requiere f válida
- **estaAndroide(a:Androide):boolean**
Requiere a ligado, retorna true si la grilla contiene un androide con la misma identidad que a.

```
10110
01110
1001
0 11
0 0
1
```

Diseño

```
GrillaAndroides
<<atributos de instancia>>
grilla [][] Androide

<<Constructores>>
GrillaAndroides(nFil,nCol: entero)
<<Comandos>>
establecerAndroide(a:Androide,
f,c:entero)
<<Consultas>>
obtenerAndroide(f,c:entero): Androide
cantFilas(): entero
cantColumnas():entero
cantAndroides():entero
cantAndroidesFila(f:entero):entero
estaAndroide(a:Androide):boolean
alMenosN(n:entero, unaNave:
NaveEspacial):boolean
alMenosNCon(n:entero,
unaNave:NaveEspacial):boolean
```

- **alMenosN(n:entero, unaNave:NaveEspacial):boolean**
Requiere n mayor a 0 y unaNave ligado, retorna true si la grilla contiene al menos n androides asociados a unaNave.
- **alMenosNCon(n:entero, unaNave:NaveEspacial):boolean**
Requiere n mayor a 0 y unaNave ligado, retorna true si la grilla contiene al menos n androides en posiciones consecutivas de una misma fila asociados a una nave equivalente a unaNave.

```
01110
1001
1 11
0 0
1
```

Implementación

```

GrillaAndroides
<<atributos de instancia>>
grilla [][] Androide

<<Constructores>>
GrillaAndroides(nFil,NCol: entero)
<<Comandos>>
establecerAndroide(a:Androide,
f,c:entero)
<<Consultas>>
obtenerAndroide(f,c:entero): Androide
cantFilas(): entero
cantColumnas():entero
cantAndroides():entero
cantAndroidesFila(f:entero):entero
estaAndroide(a:Androide):boolean
alMenosN(n:entero, unaNave:
NaveEspacial):boolean
alMenosNCon(n:entero,
unaNave:NaveEspacial):boolean
    
```

```

class GrillaAndroides{
    // ...
}
    
```

Tester alMenosN

```

GrillaAndroides
<<atributos de instancia>>
grilla [][] Androide

<<Constructores>>
GrillaAndroides(nFil,NCol: entero)
<<Comandos>>
establecerAndroide(a:Androide,
f,c:entero)
<<Consultas>>
obtenerAndroide(f,c:entero): Androide
cantFilas(): entero
cantColumnas():entero
cantAndroides():entero
cantAndroidesFila(f:entero):entero
estaAndroide(a:Androide):boolean
alMenosN(n:entero, unaNave:
NaveEspacial):boolean
alMenosNCon(n:entero,
unaNave:NaveEspacial):boolean
    
```

- El parámetro n es menor al número de columnas y los n primeros elementos de la fila 0 están asociados a unaNave.
- El parámetro n es menor al número de columnas y los n últimos elementos de la última fila están asociados a unaNave.
- El parámetro n es mayor al número de columnas y en las primeras n comparaciones los elementos están asociados a unaNave.
- La grilla contiene más de n elementos asociados a unaNave.
- La grilla contiene n-1 elementos asociados a unaNave.
- La grilla NO contiene elementos asociados a unaNave.

Otra alternativa de diseño

```

GrillaAndroides
<<atributos de instancia>>
grilla [][] Androide

<<Constructores>>
GrillaAndroides(nFil,NCol: entero)
<<Comandos>>
establecerAndroide(a:Androide,
f,c:entero): boolean
<<Consultas>>
obtenerAndroide(f,c:entero): Androide
cantFilas(): entero
cantColumnas():entero
cantAndroides():entero
cantAndroidesFila(f:entero):entero
estaAndroide(a:Androide):boolean
alMenosN(n:entero, unaNave:
NaveEspacial):boolean
alMenosNCon(n:entero,
unaNave:NaveEspacial):boolean
    
```

- GrillaAndroide(nFil,nCol: entero)**
Requiere nFil y nCol mayor a 0
- establecerAndroide(a:Androide, f,c:entero):boolean**
Si f y c representan una posición válida en la grilla, asignan el androide y retorna true, sino false
- obtenerAndroide(f,c:entero): Androide**
- cantAndroidesFila(f:entero):entero**
- estaAndroide(a:Androide):boolean**
Si el parámetro a está ligado, retorna true si la grilla contiene un androide equivalente.

```

1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1
    
```

Atención con el contrato

Cambia la signature del comando **establecerAndroide** y el contrato, el control de la posición es responsabilidad de la clase **GrillaAndroide**

- GrillaAndroide(nFil,nCol: entero)**
Requiere nFil y nCol mayor a 0
- establecerAndroide(a:Androide, f,c:entero):boolean**
Si f y c representan una posición válida en la grilla, asignan el androide y retorna true, sino false
- obtenerAndroide(f,c:entero): Androide**
- cantAndroidesFila(f:entero):entero**
- estaAndroide(a:Androide):boolean**
Si el parámetro a está ligado, retorna true si la grilla contiene un androide equivalente.

```

1 0 0 1 1
1 0 1 1 0
0 1 1 1 0
1 0 0 1
1 1 1
0 0
1
    
```

Atención con el contrato

Si el diseñador no especifica que la clase cliente se compromete a validar la posición, el control es responsabilidad del método **obtenerAndroide**. Como tampoco se indica qué retorna si la posición no es válida, decidimos retornar **null**.

- GrillaAndroide(nFil,nCol: entero)**
Requiere nFil y nCol mayor a 0
- establecerAndroide(a:Androide, f,c:entero):boolean**
Si f y c representan una posición válida en la grilla, asignan el androide y retorna true, sino false
- obtenerAndroide(f,c:entero): Androide**
- cantAndroidesFila(f:entero):entero**
- estaAndroide(a:Androide):boolean**
Si el parámetro a está ligado, retorna true si la grilla contiene un androide equivalente.
- alMenosN(n:entero, unaNave:NaveEspacial):boolean**
Requiere n mayor a 0 y nave ligado, retorna true si la grilla contiene al menos n androides asociados a unaNave.

Atención con el contrato

Si el diseñador no especifica que la clase cliente se compromete a validar que $\neq$ es una fila válida, el control es responsabilidad del método **cantAndroidesFila**. Como tampoco se indica qué retorna si la fila no es válida, decidimos retornar 0.

- GrillaAndroide(nFil,nCol: entero)**
Requiere nFil y nCol mayor a 0
- establecerAndroide(a:Androide, f,c:entero):boolean**
Si f y c representan una posición válida en la grilla, asignan el androide y retorna true, sino false
- obtenerAndroide(f,c:entero): Androide**
- cantAndroidesFila(f:entero):entero**
- estaAndroide(a:Androide):boolean**
Si el parámetro a está ligado, retorna true si la grilla contiene un androide equivalente.
- alMenosN(n:entero, unaNave:NaveEspacial):boolean**
Requiere n mayor a 0 y nave ligado, retorna true si la grilla contiene al menos n androides asociados a unaNave.

Atención con el contrato

La consulta retorna false si la grilla NO contiene un androide equivalente al parámetro formal o si el parámetro NO está ligado.

- **GrillaAndroide(nFil,nCol: entero)**
Requiere nFil y nCol mayor a 0
- **establecerAndroide(a:Androide, f,c:entero):boolean**

Si f y c representan una posición válida en la grilla, asignan el androide y retorna true, sino false

- **obtenerAndroide(f,c:entero): Androide**
- **cantAndroidesFila(f:entero):entero**
- **estaAndroide(a:Androide):boolean**

Si el parámetro a está ligado, retorna true si la grilla contiene un androide equivalente.

- **alMenosN(n:entero, unaNave:NaveEspacial):boolean**

Requiere n mayor a 0 y nave ligado, retorna true si la grilla contiene al menos n androides asociados a unaNave.

